

Database Models — b-secure Backend

All Django models across all apps, with field definitions, relationships, indexes, and the booking state machine.

Table of Contents

1. [Model Design Principles](#)
 2. [users App Models](#)
 3. [guards App Models](#)
 4. [bookings App Models](#)
 5. [tracking App Models](#)
 6. [payments App Models](#)
 7. [notifications App Models](#)
 8. [sos App Models](#)
 9. [reviews App Models](#)
 10. [analytics App Models](#)
 11. [Entity Relationship Overview](#)
 12. [Database Indexes & Performance](#)
 13. [Migrations Strategy](#)
-

1. Model Design Principles

- **UUID primary keys** on all models (not auto-increment integers) — avoids enumeration attacks and works cleanly across distributed systems.
- **created_at / updated_at** on every model via a shared `TimeStampedModel` base class.
- **Soft deletes** on User and Guard models — never hard-delete; set `is_deleted=True`.
- **PostGIS geometry fields** for guard location to enable fast proximity queries.
- **Django FSM** for booking state transitions — prevents invalid state jumps.

- `db_table` explicitly set on every model for clean SQL table names.

Base Model

```
# utils/models.py

import uuid
from django.db import models

class TimeStampedModel(models.Model):
    """Abstract base class with UUID PK and timestamps."""
    id = models.UUIDField(primary_key=True, default=uuid.uuid4,
        editable=False)
    created_at = models.DateTimeField(auto_now_add=True,
        db_index=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        abstract = True
```

2. users App Models

```
# apps/users/models.py

import uuid
from django.contrib.auth.models import AbstractBaseUser,
    PermissionsMixin, BaseUserManager
from django.db import models
from utils.models import TimeStampedModel

class UserProfileManager(BaseUserManager):
    def create_user(self, phone_number, password=None,
        **extra_fields):
        if not phone_number:
            raise ValueError('Phone number is required')
        user = self.model(phone_number=phone_number,
            **extra_fields)
        user.set_unusable_password() # b-secure uses OTP, not
            passwords
        user.save(using=self._db)
        return user

    def create_superuser(self, phone_number, password=None,
        **extra_fields):
        extra_fields.setdefault('is_staff', True)
        extra_fields.setdefault('is_superuser', True)
        extra_fields.setdefault('role', 'ADMIN')
```

```

user = self.create_user(phone_number, password,
**extra_fields)
user.set_password(password) # Admin can use password
login
user.save(using=self._db)
return user

```

```

class UserProfile(AbstractBaseUser, PermissionsMixin,
TimestampedModel):
    """
    Central user model. Used for both regular users AND guards
    (guards have an associated GuardProfile via OneToOne).
    Also used for admin/staff accounts.
    """

    ROLE_CHOICES = [
        ('USER', 'Regular User'),
        ('GUARD', 'Security Guard'),
        ('ADMIN', 'Platform Admin'),
        ('STAFF', 'Platform Staff'),
    ]

    GENDER_CHOICES = [
        ('MALE', 'Male'),
        ('FEMALE', 'Female'),
        ('OTHER', 'Other'),
        ('PREFER_NOT_TO_SAY', 'Prefer not to say'),
    ]

    # Core identity
    phone_number = models.CharField(max_length=20, unique=True,
db_index=True)
    email = models.EmailField(blank=True, null=True)
    full_name = models.CharField(max_length=150, blank=True)
    gender = models.CharField(max_length=20,
choices=GENDER_CHOICES, blank=True)
    date_of_birth = models.DateField(null=True, blank=True)
    profile_photo = models.ImageField(upload_to='users/photos/',
null=True, blank=True)

    # Role
    role = models.CharField(max_length=20, choices=ROLE_CHOICES,
default='USER')

    # Social auth identifiers
    google_id = models.CharField(max_length=128, unique=True,
null=True, blank=True)
    apple_id = models.CharField(max_length=128, unique=True,
null=True, blank=True)

    # Push notification token (updated on each app open)
    fcm_token = models.CharField(max_length=512, blank=True)

```

```

# Account status
is_active = models.BooleanField(default=True)
is_suspended = models.BooleanField(default=False)
suspension_reason = models.TextField(blank=True)
suspension_ends_at = models.DateTimeField(null=True,
    blank=True)
is_deleted = models.BooleanField(default=False) # Soft delete
deleted_at = models.DateTimeField(null=True, blank=True)

# Django auth fields
is_staff = models.BooleanField(default=False)
last_login = models.DateTimeField(null=True, blank=True)

USERNAME_FIELD = 'phone_number'
REQUIRED_FIELDS = []

objects = UserProfileManager()

class Meta:
    db_table = 'users_profile'
    indexes = [
        models.Index(fields=['role', 'is_active']),
        models.Index(fields=['is_deleted', 'created_at']),
    ]

def __str__(self):
    return f'{self.full_name or self.phone_number}'
    [{self.role}]'

@property
def is_guard(self):
    return self.role == 'GUARD'

@property
def display_name(self):
    return self.full_name or self.phone_number

class Address(TimeStampedModel):
    """Saved addresses for a user (home, office, custom)."""

    LABEL_CHOICES = [
        ('HOME', 'Home'),
        ('OFFICE', 'Office'),
        ('OTHER', 'Other'),
    ]

    user = models.ForeignKey(
        UserProfile, on_delete=models.CASCADE,
        related_name='addresses'
    )
    label = models.CharField(max_length=20,
        choices=LABEL_CHOICES, default='HOME')

```

```

custom_label = models.CharField(max_length=50, blank=True) #
    For 'OTHER'
line1 = models.CharField(max_length=255)
line2 = models.CharField(max_length=255, blank=True)
city = models.CharField(max_length=100)
state = models.CharField(max_length=100)
pincode = models.CharField(max_length=10)
country = models.CharField(max_length=50, default='India')
latitude = models.DecimalField(max_digits=9,
    decimal_places=6, null=True, blank=True)
longitude = models.DecimalField(max_digits=9,
    decimal_places=6, null=True, blank=True)
is_default = models.BooleanField(default=False)

```

class Meta:

```

    db_table = 'users_address'
    # Only one default address per user
    constraints = [
        models.UniqueConstraint(
            fields=['user', 'is_default'],
            condition=models.Q(is_default=True),
            name='unique_default_address_per_user'
        )
    ]

```

def save(self, *args, **kwargs):

```

    # If setting this as default, unset others
    if self.is_default:
        Address.objects.filter(user=self.user,
            is_default=True).update(is_default=False)
    super().save(*args, **kwargs)

```

class EmergencyContact(TimeStampedModel):

"""Emergency contacts for SOS alerts. Up to 5 per user."""

```

RELATIONSHIP_CHOICES = [
    ('SPOUSE', 'Spouse'),
    ('PARENT', 'Parent'),
    ('SIBLING', 'Sibling'),
    ('CHILD', 'Child'),
    ('FRIEND', 'Friend'),
    ('COLLEAGUE', 'Colleague'),
    ('OTHER', 'Other'),
]

```

```

user = models.ForeignKey(
    UserProfile, on_delete=models.CASCADE,
    related_name='emergency_contacts'
)
name = models.CharField(max_length=150)
phone_number = models.CharField(max_length=20)
relationship = models.CharField(

```

```

        max_length=20, choices=RELATIONSHIP_CHOICES,
        default='OTHER'
    )
    is_primary = models.BooleanField(default=False)
    is_verified = models.BooleanField(default=False) # Future:
    OTP verify contact

    class Meta:
        db_table = 'users_emergency_contact'
        # Max 5 contacts per user – enforced in serializer/service
        constraints = [
            models.UniqueConstraint(
                fields=['user', 'is_primary'],
                condition=models.Q(is_primary=True),
                name='unique_primary_emergency_contact'
            )
        ]

    def __str__(self):
        return f'{self.name} ({self.relationship}) for {self.user}'

```

3. guards App Models

```

# apps/guards/models.py

from django.db import models
from django.contrib.gis.db import models as gis_models
from utils.models import TimeStampedModel
from apps.users.models import UserProfile

class GuardProfile(TimeStampedModel):
    """
    Extended profile for security guards.
    OneToOne with UserProfile (guard's base account).
    """

    GUARD_TYPE_CHOICES = [
        ('UNARMED', 'Unarmed Security Guard'),
        ('ARMED', 'Armed Security Guard'),
        ('FEMALE', 'Female Security Guard'),
        ('CP0', 'Close Protection Officer'),
        ('EVENT', 'Event Security'),
        ('K9', 'K9 / Dog Handler'),
    ]

    VERIFICATION_STATUS_CHOICES = [
        ('PENDING', 'Pending – Documents not yet uploaded'),
        ('UNDER_REVIEW', 'Under Review – Admin reviewing documents'),
    ]

```

```

        ('ACTIVE', 'Active – Verified and can accept bookings'),
        ('SUSPENDED', 'Suspended – Temporarily deactivated'),
        ('BANNED', 'Banned – Permanently deactivated'),
        ('DOCUMENTS_REJECTED', 'Documents Rejected – Needs
        resubmission'),
    ]

    user = models.OneToOneField(
        UserProfile, on_delete=models.CASCADE,
        related_name='guard_profile'
    )

    # Professional details
    guard_type = models.CharField(
        max_length=20, choices=GUARD_TYPE_CHOICES,
        default='UNARMED'
    )
    years_of_experience =
        models.PositiveSmallIntegerField(default=0)
    bio = models.TextField(blank=True, max_length=500)
    languages_spoken = models.JSONField(default=list) #
        ['Hindi', 'English', 'Kannada']
    skills = models.JSONField(default=list) #
        ['crowd_control', 'first_aid']

    # Verification
    verification_status = models.CharField(
        max_length=25, choices=VERIFICATION_STATUS_CHOICES,
        default='PENDING'
    )
    verified_at = models.DateTimeField(null=True, blank=True)
    verified_by = models.ForeignKey(
        UserProfile, on_delete=models.SET_NULL, null=True,
        blank=True,
        related_name='guards_verified',
        limit_choices_to={'is_staff': True}
    )

    # Live location (PostGIS Point field for efficient geospatial
    queries)
    current_location = gis_models.PointField(
        geography=True, null=True, blank=True,
        help_text='Current GPS coordinates. Updated every 3–5s
        when on duty.'
    )
    last_location_update = models.DateTimeField(null=True,
        blank=True)
    is_online = models.BooleanField(default=False, db_index=True)

    # Ratings (denormalized for fast queries – updated by signal
    on new review)
    average_rating = models.DecimalField(
        max_digits=3, decimal_places=2, default=0.00
    )
    total_reviews = models.PositiveIntegerField(default=0)

```

```

total_sessions_completed =
    models.PositiveIntegerField(default=0)

# Payout details
bank_account_number = models.CharField(max_length=20,
    blank=True)
bank_ifsc_code = models.CharField(max_length=11, blank=True)
upi_id = models.CharField(max_length=50, blank=True)
payout_preference = models.CharField(
    max_length=10,
    choices=[('BANK', 'Bank Transfer'), ('UPI', 'UPI')],
    default='UPI'
)

# Working preferences
preferred_work_radius_km =
    models.PositiveSmallIntegerField(default=10)
max_daily_hours = models.PositiveSmallIntegerField(default=12)

class Meta:
    db_table = 'guards_profile'
    indexes = [
        models.Index(fields=['verification_status',
            'is_online']),
        models.Index(fields=['guard_type', 'is_online']),
        models.Index(fields=['average_rating']),
    ]

def __str__(self):
    return f'Guard: {self.user.display_name}
    [{self.guard_type}] - {self.verification_status}'

@property
def is_available(self):
    return self.is_online and self.verification_status ==
        'ACTIVE'

class GuardDocument(TimeStampedModel):
    """
    Documents uploaded by guards for verification.
    Each document type can be uploaded once; re-upload replaces
    previous.
    """

    DOCUMENT_TYPE_CHOICES = [
        ('GOVT_ID', 'Government Photo ID (Aadhaar / Passport /
            Voter ID)'),
        ('POLICE_CERT', 'Police Verification Certificate'),
        ('PSARA_LICENSE', 'PSARA Security Guard License'),
        ('TRAINING_CERT', 'Security Training Certificate'),
        ('ARMED_LICENSE', 'Arms License (Armed Guards Only)'),
        ('PROFILE_PHOTO', 'Live Profile Selfie'),
        ('ADDRESS_PROOF', 'Address Proof'),
    ]

```



```

]

DOCUMENT_STATUS_CHOICES = [
    ('UPLOADED', 'Uploaded – Awaiting review'),
    ('UNDER_REVIEW', 'Under Review'),
    ('APPROVED', 'Approved'),
    ('REJECTED', 'Rejected – Needs resubmission'),
    ('EXPIRED', 'Expired – Renewal required'),
]

guard = models.ForeignKey(
    GuardProfile, on_delete=models.CASCADE,
    related_name='documents'
)
document_type = models.CharField(max_length=25,
    choices=DOCUMENT_TYPE_CHOICES)
file = models.FileField(upload_to='guards/documents/') #
    Stored in S3 (private)
file_name = models.CharField(max_length=255)
status = models.CharField(
    max_length=15, choices=DOCUMENT_STATUS_CHOICES,
    default='UPLOADED', db_index=True
)

# Expiry tracking
expiry_date = models.DateField(null=True, blank=True)
expiry_reminder_sent = models.BooleanField(default=False)

# Admin review
reviewed_by = models.ForeignKey(
    'users.UserProfile', on_delete=models.SET_NULL,
    null=True, blank=True,
    related_name='documents_reviewed',
    limit_choices_to={'is_staff': True}
)
review_notes = models.TextField(blank=True)
reviewed_at = models.DateTimeField(null=True, blank=True)

class Meta:
    db_table = 'guards_document'
    # One document per type per guard (latest replaces old)
    unique_together = [('guard', 'document_type')]

def __str__(self):
    return f'{self.get_document_type_display()} – {self.guard} [{self.status}]'

class GuardAvailability(TimeStampedModel):
    """
    Weekly recurring availability schedule for a guard.
    One record per weekday. Used for advance booking scheduling.
    """

```

```

WEEKDAY_CHOICES = [
    (0, 'Monday'), (1, 'Tuesday'), (2, 'Wednesday'),
    (3, 'Thursday'), (4, 'Friday'), (5, 'Saturday'), (6,
    'Sunday'),
]

guard = models.ForeignKey(
    GuardProfile, on_delete=models.CASCADE,
    related_name='availability_schedule'
)
weekday =
    models.PositiveSmallIntegerField(choices=WEEKDAY_CHOICES)
start_time = models.TimeField()
end_time = models.TimeField()
is_available = models.BooleanField(default=True)

class Meta:
    db_table = 'guards_availability'
    unique_together = [('guard', 'weekday')]
    ordering = ['weekday', 'start_time']

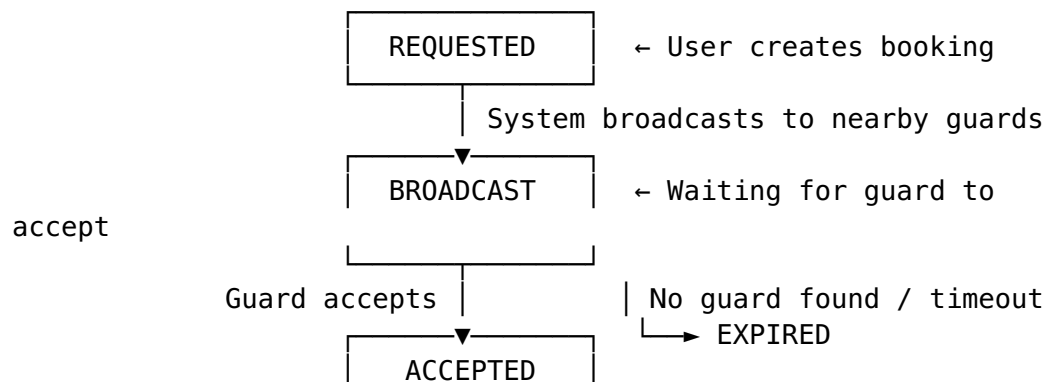
class GuardBlackoutDate(TimeStampedModel):
    """Specific dates when a guard is unavailable."""
    guard = models.ForeignKey(
        GuardProfile, on_delete=models.CASCADE,
        related_name='blackout_dates'
    )
    date = models.DateField()
    reason = models.CharField(max_length=200, blank=True)

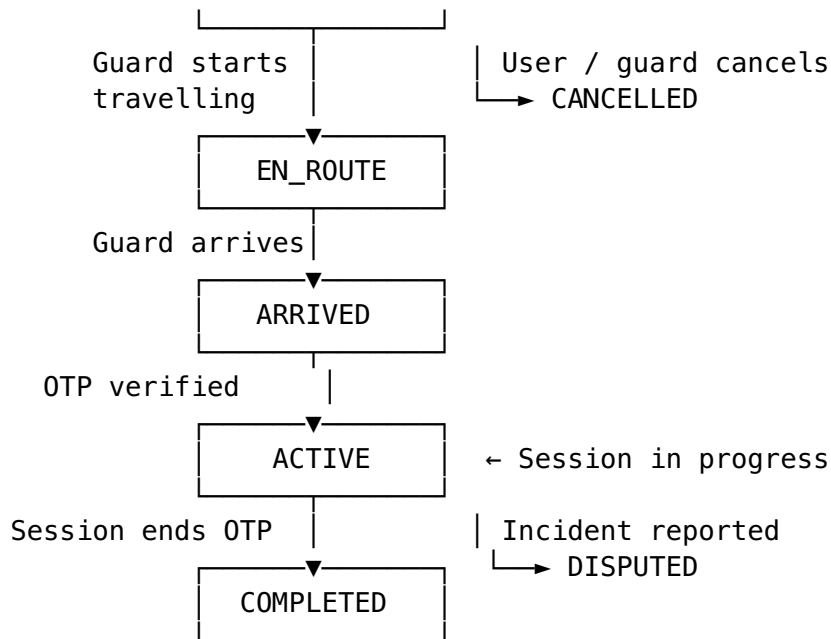
class Meta:
    db_table = 'guards_blackout_date'
    unique_together = [('guard', 'date')]

```

4. bookings App Models

Booking State Machine





```
# apps/bookings/models.py
```

```
from django.db import models
from django.utils import timezone
from django_fsm import FSMField, transition
from utils.models import TimeStampedModel
from apps.users.models import UserProfile
from apps.guards.models import GuardProfile
import hashlib, secrets
```

```
class Booking(TimeStampedModel):
    """
    Core booking model. Represents a single security service
    engagement.
    Uses django-fsm for state transition management.
    """

    SERVICE_TYPE_CHOICES = [
        ('HOURLY', 'Hourly Protection'),
        ('DAILY', 'Daily Protection (8h or 12h shift)'),
        ('WEEKLY', 'Weekly Protection (5 or 7 days)'),
        ('MONTHLY', 'Monthly Protection (30 days)'),
    ]

    GUARD_TYPE_CHOICES = [
        ('UNARMED', 'Unarmed Guard'),
        ('ARMED', 'Armed Guard'),
        ('FEMALE', 'Female Guard'),
        ('CP0', 'Close Protection Officer'),
        ('EVENT', 'Event Security'),
    ]

    STATUS_CHOICES = [
```

```

        ('REQUESTED', 'Requested'),
        ('BROADCAST', 'Broadcasting to Guards'),
        ('ACCEPTED', 'Guard Accepted'),
        ('EN_ROUTE', 'Guard En Route'),
        ('ARRIVED', 'Guard Arrived'),
        ('ACTIVE', 'Session Active'),
        ('COMPLETED', 'Session Completed'),
        ('CANCELLED', 'Cancelled'),
        ('DISPUTED', 'Disputed'),
        ('EXPIRED', 'Expired – No Guard Found'),
    ]

# Participants
user = models.ForeignKey(
    UserProfile, on_delete=models.PROTECT,
    related_name='bookings'
)
guard = models.ForeignKey(
    GuardProfile, on_delete=models.PROTECT,
    null=True, blank=True, related_name='bookings'
)

# Service details
service_type = models.CharField(max_length=10,
    choices=SERVICE_TYPE_CHOICES)
guard_type_requested = models.CharField(max_length=10,
    choices=GUARD_TYPE_CHOICES)

# State (managed by django-fsm)
status = FSMField(default='REQUESTED',
    choices=STATUS_CHOICES, db_index=True)

# Scheduling
scheduled_start = models.DateTimeField()
scheduled_end = models.DateTimeField()
is_immediate = models.BooleanField(default=True) # False =
    advance booking

# Session timestamps (filled in as session progresses)
guard_accepted_at = models.DateTimeField(null=True,
    blank=True)
guard_arrived_at = models.DateTimeField(null=True, blank=True)
session_started_at = models.DateTimeField(null=True,
    blank=True)
session_ended_at = models.DateTimeField(null=True, blank=True)

# Location (where service is required)
service_address = models.TextField()
service_latitude = models.DecimalField(max_digits=9,
    decimal_places=6)
service_longitude = models.DecimalField(max_digits=9,
    decimal_places=6)

# Pricing

```

```

base_rate_per_hour = models.DecimalField(max_digits=8,
    decimal_places=2)
surge_multiplier = models.DecimalField(max_digits=4,
    decimal_places=2, default=1.00)
promo_discount = models.DecimalField(max_digits=8,
    decimal_places=2, default=0.00)
total_amount = models.DecimalField(max_digits=10,
    decimal_places=2, null=True, blank=True)
platform_fee = models.DecimalField(max_digits=8,
    decimal_places=2, null=True, blank=True)
guard_earnings = models.DecimalField(max_digits=8,
    decimal_places=2, null=True, blank=True)
tax_amount = models.DecimalField(max_digits=8,
    decimal_places=2, null=True, blank=True)

# OTP for session start/end verification
start_otp_hash = models.CharField(max_length=64, blank=True)
end_otp_hash = models.CharField(max_length=64, blank=True)

# Cancellation
cancelled_by = models.ForeignKey(
    UserProfile, on_delete=models.SET_NULL, null=True,
    blank=True,
    related_name='cancelled_bookings'
)
cancellation_reason = models.TextField(blank=True)
cancelled_at = models.DateTimeField(null=True, blank=True)

# Recurring booking support
is_recurring = models.BooleanField(default=False)
recurrence_rule = models.JSONField(null=True, blank=True) #
    iCal RRULE-style
parent_booking = models.ForeignKey(
    'self', on_delete=models.SET_NULL, null=True, blank=True,
    related_name='recurrence_instances'
)

# Admin
admin_notes = models.TextField(blank=True)
is_flagged = models.BooleanField(default=False)

class Meta:
    db_table = 'bookings_booking'
    indexes = [
        models.Index(fields=['user', 'status']),
        models.Index(fields=['guard', 'status']),
        models.Index(fields=['status', 'scheduled_start']),
        models.Index(fields=['is_immediate', 'status']),
    ]
    ordering = ['-created_at']

# --- FSM Transitions ---

```

```

@transition(field=status, source='REQUESTED',
            target='BROADCAST')
def start_broadcast(self):
    """System starts broadcasting booking request to nearby
    guards."""
    pass

@transition(field=status, source='BROADCAST',
            target='ACCEPTED')
def guard_accept(self, guard: GuardProfile):
    """A guard accepts the booking."""
    self.guard = guard
    self.guard_accepted_at = timezone.now()

@transition(field=status, source='ACCEPTED',
            target='EN_ROUTE')
def guard_start_travel(self):
    """Guard begins travelling to client location."""
    pass

@transition(field=status, source='EN_ROUTE', target='ARRIVED')
def guard_arrive(self):
    """Guard marks themselves as arrived at client
    location."""
    self.guard_arrived_at = timezone.now()

@transition(field=status, source='ARRIVED', target='ACTIVE')
def start_session(self):
    """OTP verified – session officially starts."""
    self.session_started_at = timezone.now()

@transition(field=status, source='ACTIVE', target='COMPLETED')
def complete_session(self):
    """OTP verified – session ends normally."""
    self.session_ended_at = timezone.now()

@transition(field=status, source='ACTIVE', target='DISPUTED')
def dispute_session(self):
    """User or guard raises a dispute."""
    pass

@transition(
    field=status,
    source=['REQUESTED', 'BROADCAST', 'ACCEPTED', 'EN_ROUTE',
           'ARRIVED'],
    target='CANCELLED'
)
def cancel(self, cancelled_by: UserProfile, reason: str = ''):
    """Cancel booking (allowed up to session start)."""
    self.cancelled_by = cancelled_by
    self.cancellation_reason = reason
    self.cancelled_at = timezone.now()

```

```

@transition(field=status, source='BROADCAST',
            target='EXPIRED')
def expire(self):
    """No guard found within timeout window."""
    pass

# --- OTP helpers ---

def generate_start_otp(self) -> str:
    otp = str(secrets.randbelow(10000)).zfill(4)
    self.start_otp_hash =
        hashlib.sha256(otp.encode()).hexdigest()
    self.save(update_fields=['start_otp_hash'])
    return otp

def verify_start_otp(self, otp: str) -> bool:
    return self.start_otp_hash ==
        hashlib.sha256(otp.encode()).hexdigest()

def generate_end_otp(self) -> str:
    otp = str(secrets.randbelow(10000)).zfill(4)
    self.end_otp_hash =
        hashlib.sha256(otp.encode()).hexdigest()
    self.save(update_fields=['end_otp_hash'])
    return otp

def verify_end_otp(self, otp: str) -> bool:
    return self.end_otp_hash ==
        hashlib.sha256(otp.encode()).hexdigest()

def __str__(self):
    return f'Booking {self.id} - {self.user} + {self.guard} '
        f'[{self.status}]'

class BookingBroadcast(TimeStampedModel):
    """
    Tracks which guards received a booking request broadcast
    and their response (accepted, declined, timed out).
    Used for analytics and re-broadcast logic.
    """

    RESPONSE_CHOICES = [
        ('SENT', 'Request Sent'),
        ('ACCEPTED', 'Accepted'),
        ('DECLINED', 'Declined'),
        ('TIMEOUT', 'No Response – Timed Out'),
    ]

    booking = models.ForeignKey(
        Booking, on_delete=models.CASCADE,
        related_name='broadcasts'
    )
    guard = models.ForeignKey(

```

```

        GuardProfile, on_delete=models.CASCADE,
        related_name='broadcast_requests'
    )
    response = models.CharField(max_length=10,
                                choices=RESPONSE_CHOICES, default='SENT')
    broadcast_radius_km = models.PositiveSmallIntegerField()
    sent_at = models.DateTimeField(auto_now_add=True)
    responded_at = models.DateTimeField(null=True, blank=True)

    class Meta:
        db_table = 'bookings_broadcast'
        unique_together = [('booking', 'guard')]

class GuardCheckIn(TimeStampedModel):
    """
    Records guard check-ins during long active sessions.
    Missed check-ins trigger alerts to user and admin.
    """
    booking = models.ForeignKey(
        Booking, on_delete=models.CASCADE, related_name='checkins'
    )
    guard = models.ForeignKey(
        GuardProfile, on_delete=models.CASCADE,
        related_name='checkins'
    )
    latitude = models.DecimalField(max_digits=9, decimal_places=6)
    longitude = models.DecimalField(max_digits=9,
                                    decimal_places=6)
    notes = models.CharField(max_length=200, blank=True)
    is_auto = models.BooleanField(default=False)
    # True if system auto-checked

    class Meta:
        db_table = 'bookings_checkin'
        ordering = ['-created_at']

```

5. tracking App Models

```

# apps/tracking/models.py

from django.db import models
from django.contrib.gis.db import models as gis_models
from utils.models import TimeStampedModel

class LocationSnapshot(TimeStampedModel):
    """
    Time-series location data for guards during active sessions.
    High write volume – consider TimescaleDB extension for
    PostgreSQL
    """

```


or periodic archival of old records.

Retention policy: 90 days raw, then purged.

```
"""

booking = models.ForeignKey(
    'bookings.Booking', on_delete=models.CASCADE,
    related_name='location_snapshots'
)
guard = models.ForeignKey(
    'guards.GuardProfile', on_delete=models.CASCADE,
    related_name='location_history'
)

# PostGIS point for potential geospatial analysis
location = gis_models.PointField(geography=True)
accuracy_meters = models.FloatField(null=True, blank=True)
speed_kmh = models.FloatField(null=True, blank=True)
bearing_degrees = models.FloatField(null=True, blank=True)

# Timestamp is indexed for fast time-range queries (session
# replay)
timestamp = models.DateTimeField(db_index=True)

class Meta:
    db_table = 'tracking_location_snapshot'
    # Composite index for efficient session replay queries
    indexes = [
        models.Index(fields=['booking', 'timestamp']),
        models.Index(fields=['guard', 'timestamp']),
    ]
    ordering = ['timestamp']

def __str__(self):
    return f'Location snapshot for booking {self.booking_id} '
    at {self.timestamp}'
```

6. payments App Models

apps/payments/models.py

```
from django.db import models
from django.core.validators import MinValueValidator
from utils.models import TimeStampedModel

class Wallet(TimeStampedModel):
    """
    In-app wallet for users. Guards have separate earnings tracked
    via Payout model.
    """
```

```

user = models.OneToOneField(
    'users.UserProfile', on_delete=models.CASCADE,
    related_name='wallet'
)
balance = models.DecimalField(
    max_digits=10, decimal_places=2,
    default=0.00,
    validators=[MinValueValidator(0)]
)
# Locked amount: held for active bookings awaiting completion
locked_balance = models.DecimalField(max_digits=10,
    decimal_places=2, default=0.00)

class Meta:
    db_table = 'payments_wallet'

@property
def available_balance(self):
    return self.balance - self.locked_balance

def __str__(self):
    return f'Wallet of {self.user} - ₹{self.balance}'

class Transaction(TimeStampedModel):
    """
    Immutable ledger of all wallet movements.
    Never update a transaction – only create new ones.
    """

    TRANSACTION_TYPE_CHOICES = [
        ('TOPUP', 'Wallet Top-up'),
        ('BOOKING_DEBIT', 'Booking Payment'),
        ('BOOKING_LOCK', 'Amount Locked for Booking'),
        ('BOOKING_UNLOCK', 'Amount Unlocked (Cancellation)'),
        ('REFUND', 'Refund'),
        ('PROMO_CREDIT', 'Promotional Credit'),
        ('REFERRAL_BONUS', 'Referral Bonus'),
        ('ADMIN_CREDIT', 'Admin Manual Credit'),
        ('ADMIN_DEBIT', 'Admin Manual Debit'),
    ]

    STATUS_CHOICES = [
        ('PENDING', 'Pending'),
        ('SUCCESS', 'Success'),
        ('FAILED', 'Failed'),
        ('REFUNDED', 'Refunded'),
    ]

    wallet = models.ForeignKey(
        Wallet, on_delete=models.PROTECT,
        related_name='transactions'
    )

```

```

transaction_type = models.CharField(max_length=20,
    choices=TRANSACTION_TYPE_CHOICES)
amount = models.DecimalField(max_digits=10,
    decimal_places=2) # Always positive
balance_before = models.DecimalField(max_digits=10,
    decimal_places=2)
balance_after = models.DecimalField(max_digits=10,
    decimal_places=2)
status = models.CharField(max_length=10,
    choices=STATUS_CHOICES, default='PENDING')

# Linked entities
booking = models.ForeignKey(
    'bookings.Booking', on_delete=models.SET_NULL,
    null=True, blank=True, related_name='transactions'
)

# Payment gateway references
gateway = models.CharField(
    max_length=20,
    choices=[('RAZORPAY', 'Razorpay'), ('STRIPE', 'Stripe'),
    ('INTERNAL', 'Internal')],
    default='INTERNAL'
)
gateway_order_id = models.CharField(max_length=100,
    blank=True)
gateway_payment_id = models.CharField(max_length=100,
    blank=True)
gateway_signature = models.CharField(max_length=256,
    blank=True)

description = models.CharField(max_length=255, blank=True)
admin_note = models.TextField(blank=True)

class Meta:
    db_table = 'payments_transaction'
    indexes = [
        models.Index(fields=['wallet', 'created_at']),
        models.Index(fields=['booking', 'transaction_type']),
        models.Index(fields=['gateway_payment_id']),
        models.Index(fields=['status', 'transaction_type']),
    ]
    ordering = ['-created_at']

class PaymentOrder(TimeStampedModel):
    """
    Represents a payment initiation with a payment gateway.
    Created before payment; updated via webhook on completion.
    """

    STATUS_CHOICES = [
        ('CREATED', 'Created at Gateway'),
        ('ATTEMPTED', 'Payment Attempted'),
    ]

```

```

        ('PAID', 'Successfully Paid'),
        ('FAILED', 'Payment Failed'),
        ('EXPIRED', 'Order Expired'),
    ]

    user = models.ForeignKey(
        'users.UserProfile', on_delete=models.PROTECT,
        related_name='payment_orders'
    )
    amount = models.DecimalField(max_digits=10,
        decimal_places=2) # Amount in INR
    currency = models.CharField(max_length=3, default='INR')
    purpose = models.CharField(
        max_length=20,
        choices=[('WALLET_TOPUP', 'Wallet Top-up'), ('BOOKING',
            'Direct Booking Payment')],
        default='WALLET_TOPUP'
    )
    status = models.CharField(max_length=10,
        choices=STATUS_CHOICES, default='CREATED')
    gateway = models.CharField(max_length=20,
        choices=[('RAZORPAY', 'Razorpay'), ('STRIPE', 'Stripe')])
    gateway_order_id = models.CharField(max_length=100,
        unique=True)
    gateway_response = models.JSONField(null=True, blank=True) #
        Full gateway response
    booking = models.ForeignKey(
        'bookings.Booking', on_delete=models.SET_NULL, null=True,
        blank=True
    )

    class Meta:
        db_table = 'payments_order'
        indexes = [
            models.Index(fields=['user', 'status']),
            models.Index(fields=['gateway_order_id']),
        ]

```

```

class Payout(TimeStampedModel):
    """
    Tracks earnings payouts to guards.
    Created per payout cycle (daily or weekly).
    """

```

```

STATUS_CHOICES = [
    ('PENDING', 'Pending Approval'),
    ('PROCESSING', 'Processing Transfer'),
    ('COMPLETED', 'Transfer Completed'),
    ('FAILED', 'Transfer Failed'),
    ('ON_HOLD', 'On Hold – Under Review'),
]

```

```

guard = models.ForeignKey(

```

```

        'guards.GuardProfile', on_delete=models.PROTECT,
        related_name='payouts'
    )
    amount = models.DecimalField(max_digits=10, decimal_places=2)
    status = models.CharField(max_length=15,
                              choices=STATUS_CHOICES, default='PENDING')
    period_start = models.DateField()
    period_end = models.DateField()

    # Razorpay Payouts API
    razorpay_payout_id = models.CharField(max_length=100,
                                           blank=True)
    bank_reference = models.CharField(max_length=100, blank=True)

    failure_reason = models.TextField(blank=True)
    processed_at = models.DateTimeField(null=True, blank=True)
    processed_by = models.ForeignKey(
        'users.UserProfile', on_delete=models.SET_NULL,
        null=True, blank=True, related_name='processed_payouts',
        limit_choices_to={'is_staff': True}
    )

    # Sessions included in this payout
    bookings = models.ManyToManyField('bookings.Booking',
                                      blank=True, related_name='payouts')

    class Meta:
        db_table = 'payments_payout'
        indexes = [
            models.Index(fields=['guard', 'status']),
            models.Index(fields=['status', 'period_start']),
        ]

```

7. notifications App Models

```

# apps/notifications/models.py

from django.db import models
from utils.models import TimeStampedModel

class NotificationLog(TimeStampedModel):
    """Records every notification sent through the platform."""

    CHANNEL_CHOICES = [
        ('PUSH', 'Push Notification (FCM)'),
        ('SMS', 'SMS'),
        ('EMAIL', 'Email'),
        ('IN_APP', 'In-App Notification'),
        ('WHATSAPP', 'WhatsApp (future)'),
    ]

```

```

STATUS_CHOICES = [
    ('QUEUED', 'Queued in Celery'),
    ('SENT', 'Sent to Provider'),
    ('DELIVERED', 'Delivered to Device'),
    ('FAILED', 'Failed to Send'),
    ('BOUNCED', 'Bounced / Invalid'),
]

recipient = models.ForeignKey(
    'users.UserProfile', on_delete=models.CASCADE,
    related_name='notifications'
)
channel = models.CharField(max_length=10,
    choices=CHANNEL_CHOICES)
notification_type = models.CharField(max_length=50) # e.g.
    'GUARD_ASSIGNED'
title = models.CharField(max_length=255, blank=True)
body = models.TextField()
data = models.JSONField(default=dict) # Extra data sent
    with push notification
status = models.CharField(max_length=10,
    choices=STATUS_CHOICES, default='QUEUED')
provider_message_id = models.CharField(max_length=255,
    blank=True)
failure_reason = models.TextField(blank=True)
is_read = models.BooleanField(default=False) # For in-app
    notifications
read_at = models.DateTimeField(null=True, blank=True)

class Meta:
    db_table = 'notifications_log'
    indexes = [
        models.Index(fields=['recipient', 'is_read',
            'created_at']),
        models.Index(fields=['notification_type', 'status']),
    ]
    ordering = ['-created_at']

class NotificationPreference(TimeStampedModel):
    """Per-user notification preferences per channel per event
    type."""
    user = models.OneToOneField(
        'users.UserProfile', on_delete=models.CASCADE,
        related_name='notification_preferences'
    )
    push_enabled = models.BooleanField(default=True)
    sms_enabled = models.BooleanField(default=True)
    email_enabled = models.BooleanField(default=True)
    marketing_push = models.BooleanField(default=False) # Opt-in
        for promos
    marketing_email = models.BooleanField(default=False)

```

```
class Meta:
    db_table = 'notifications_preference'
```

8. sos App Models

```
# apps/sos/models.py
```

```
from django.db import models
from utils.models import TimeStampedModel
```

```
class SOSAlert(TimeStampedModel):
    """
    Records every SOS trigger event.
    Mission-critical model – writes must be synchronous (not
    async/Celery).
    """

    TRIGGER_METHOD_CHOICES = [
        ('BUTTON', 'Manual Button Press'),
        ('SHAKE', 'Shake Gesture'),
        ('AUTO_CHECKIN', 'Auto – Missed Check-in Escalation'),
        ('GUARD_OFFLINE', 'Auto – Guard Went Offline During
        Session'),
        ('GUARD_DISTRESS', 'Guard Distress Button'),
    ]

    STATUS_CHOICES = [
        ('TRIGGERED', 'Triggered – Awaiting Acknowledgement'),
        ('ACKNOWLEDGED', 'Acknowledged by Control Room'),
        ('RESPONDING', 'Response Team Dispatched'),
        ('RESOLVED', 'Resolved'),
        ('FALSE_ALARM', 'Resolved – False Alarm'),
    ]

    user = models.ForeignKey(
        'users.UserProfile', on_delete=models.PROTECT,
        related_name='sos_alerts'
    )
    booking = models.ForeignKey(
        'bookings.Booking', on_delete=models.SET_NULL,
        null=True, blank=True, related_name='sos_alerts'
    )
    trigger_method = models.CharField(max_length=20,
        choices=TRIGGER_METHOD_CHOICES)
    status = models.CharField(max_length=15,
        choices=STATUS_CHOICES, default='TRIGGERED',
        db_index=True)

    # Location at time of trigger
    latitude = models.DecimalField(max_digits=9, decimal_places=6)
```

```

longitude = models.DecimalField(max_digits=9,
    decimal_places=6)
location_accuracy_meters = models.FloatField(null=True,
    blank=True)

# Response tracking
assigned_to = models.ForeignKey(
    'users.UserProfile', on_delete=models.SET_NULL,
    null=True, blank=True, related_name='assigned_sos_alerts',
    limit_choices_to={'is_staff': True}
)
acknowledged_at = models.DateTimeField(null=True, blank=True)
resolved_at = models.DateTimeField(null=True, blank=True)
resolution_notes = models.TextField(blank=True)

# Audio/recording if enabled
recording_file = models.FileField(upload_to='sos/
    recordings/', null=True, blank=True)

class Meta:
    db_table = 'sos_alert'
    indexes = [
        models.Index(fields=['status', 'created_at']),
        models.Index(fields=['user', 'created_at']),
    ]
    ordering = ['-created_at']

class EmergencyContactAlert(TimeStampedModel):
    """Tracks which emergency contacts were notified for an SOS."""
    sos_alert = models.ForeignKey(
        SOSAlert, on_delete=models.CASCADE,
        related_name='contact_alerts'
    )
    contact_name = models.CharField(max_length=150)
    contact_phone = models.CharField(max_length=20)
    sms_sent = models.BooleanField(default=False)
    sms_delivered = models.BooleanField(default=False)
    call_attempted = models.BooleanField(default=False)

    class Meta:
        db_table = 'sos_emergency_contact_alert'

class Incident(TimeStampedModel):
    """
    User or guard-filed incident reports.
    Separate from SOS – incidents are retrospective reports.
    """

    INCIDENT_TYPE_CHOICES = [
        ('GUARD_MISCONDUCT', 'Guard Misconduct'),

```



```

        ('GUARD_NO_SHOW', 'Guard Did Not Arrive'),
        ('GUARD_EARLY_DEPARTURE', 'Guard Left Early'),
        ('THREATENING_BEHAVIOUR', 'Threatening Behaviour'),
        ('THEFT', 'Theft'),
        ('PROPERTY_DAMAGE', 'Property Damage'),
        ('DANGEROUS_CLIENT', 'Dangerous Client (Guard Report)'),
        ('SAFETY_CONCERN', 'General Safety Concern'),
        ('OTHER', 'Other'),
    ]

    SEVERITY_CHOICES = [
        ('LOW', 'Low'),
        ('MEDIUM', 'Medium'),
        ('HIGH', 'High'),
        ('CRITICAL', 'Critical'),
    ]

    STATUS_CHOICES = [
        ('OPEN', 'Open – Under Investigation'),
        ('IN_REVIEW', 'In Review'),
        ('RESOLVED', 'Resolved'),
        ('CLOSED', 'Closed – No Action'),
    ]

    booking = models.ForeignKey(
        'bookings.Booking', on_delete=models.PROTECT,
        related_name='incidents'
    )
    filed_by = models.ForeignKey(
        'users.UserProfile', on_delete=models.PROTECT,
        related_name='filed_incidents'
    )
    incident_type = models.CharField(max_length=30,
                                     choices=INCIDENT_TYPE_CHOICES)
    severity = models.CharField(max_length=10,
                               choices=SEVERITY_CHOICES, default='MEDIUM')
    description = models.TextField()
    status = models.CharField(max_length=15,
                             choices=STATUS_CHOICES, default='OPEN', db_index=True)

    # Admin handling
    assigned_to = models.ForeignKey(
        'users.UserProfile', on_delete=models.SET_NULL,
        null=True, blank=True, related_name='assigned_incidents',
        limit_choices_to={'is_staff': True}
    )
    resolution_notes = models.TextField(blank=True)
    resolved_at = models.DateTimeField(null=True, blank=True)

    class Meta:
        db_table = 'sos_incident'
        ordering = ['-created_at']

```

```

class IncidentEvidence(TimeStampedModel):
    """Photo or video evidence attached to an incident report."""
    incident = models.ForeignKey(
        Incident, on_delete=models.CASCADE,
        related_name='evidence'
    )
    file = models.FileField(upload_to='incidents/evidence/')
    file_type = models.CharField(max_length=10,
        choices=[('IMAGE', 'Image'), ('VIDEO', 'Video')])
    description = models.CharField(max_length=255, blank=True)

    class Meta:
        db_table = 'sos_incident_evidence'

```

9. reviews App Models

```

# apps/reviews/models.py

from django.db import models
from django.core.validators import MinValueValidator,
    MaxValueValidator
from utils.models import TimeStampedModel

class Review(TimeStampedModel):
    """
    Post-session rating and review.
    One review per completed booking.
    Guard's average_rating on GuardProfile is updated via signal.
    """

    booking = models.OneToOneField(
        'bookings.Booking', on_delete=models.PROTECT,
        related_name='review'
    )
    reviewer = models.ForeignKey(
        'users.UserProfile', on_delete=models.PROTECT,
        related_name='reviews_given'
    )
    guard = models.ForeignKey(
        'guards.GuardProfile', on_delete=models.PROTECT,
        related_name='reviews_received'
    )

    # Ratings (1-5)
    overall_rating = models.PositiveSmallIntegerField(
        validators=[MinValueValidator(1), MaxValueValidator(5)]
    )
    punctuality_rating = models.PositiveSmallIntegerField(
        null=True, blank=True,

```

```

        validators=[MinValueValidator(1), MaxValueValidator(5)]
    )
    professionalism_rating = models.PositiveSmallIntegerField(
        null=True, blank=True,
        validators=[MinValueValidator(1), MaxValueValidator(5)]
    )
    communication_rating = models.PositiveSmallIntegerField(
        null=True, blank=True,
        validators=[MinValueValidator(1), MaxValueValidator(5)]
    )
    alertness_rating = models.PositiveSmallIntegerField(
        null=True, blank=True,
        validators=[MinValueValidator(1), MaxValueValidator(5)]
    )

    comment = models.TextField(blank=True, max_length=1000)

    # Moderation
    is_flagged = models.BooleanField(default=False)
    flag_reason = models.CharField(max_length=255, blank=True)
    is_hidden = models.BooleanField(default=False) # Admin can
        hide abusive reviews

    class Meta:
        db_table = 'reviews_review'
        ordering = ['-created_at']

    def __str__(self):
        return f'Review for {self.guard} - {self.overall_rating}★'

```

10. analytics App Models

```

# apps/analytics/models.py

from django.db import models

class DailyStats(models.Model):
    """
    Pre-aggregated daily statistics.
    Populated nightly by Celery beat task.
    Used by admin dashboard to serve instant stats without heavy
        queries.
    """

    date = models.DateField(unique=True, db_index=True)

    # Bookings
    total_bookings = models.PositiveIntegerField(default=0)
    completed_bookings = models.PositiveIntegerField(default=0)

```

```

cancelled_bookings = models.PositiveIntegerField(default=0)
disputed_bookings = models.PositiveIntegerField(default=0)
hourly_bookings = models.PositiveIntegerField(default=0)
daily_bookings = models.PositiveIntegerField(default=0)
weekly_bookings = models.PositiveIntegerField(default=0)
monthly_bookings = models.PositiveIntegerField(default=0)

# Revenue
gross_revenue = models.DecimalField(max_digits=12,
    decimal_places=2, default=0)
platform_fees_collected = models.DecimalField(max_digits=12,
    decimal_places=2, default=0)
guard_earnings_paid = models.DecimalField(max_digits=12,
    decimal_places=2, default=0)
refunds_issued = models.DecimalField(max_digits=12,
    decimal_places=2, default=0)

# Users & Guards
new_users = models.PositiveIntegerField(default=0)
active_users = models.PositiveIntegerField(default=0)
new_guards = models.PositiveIntegerField(default=0)
active_guards = models.PositiveIntegerField(default=0)

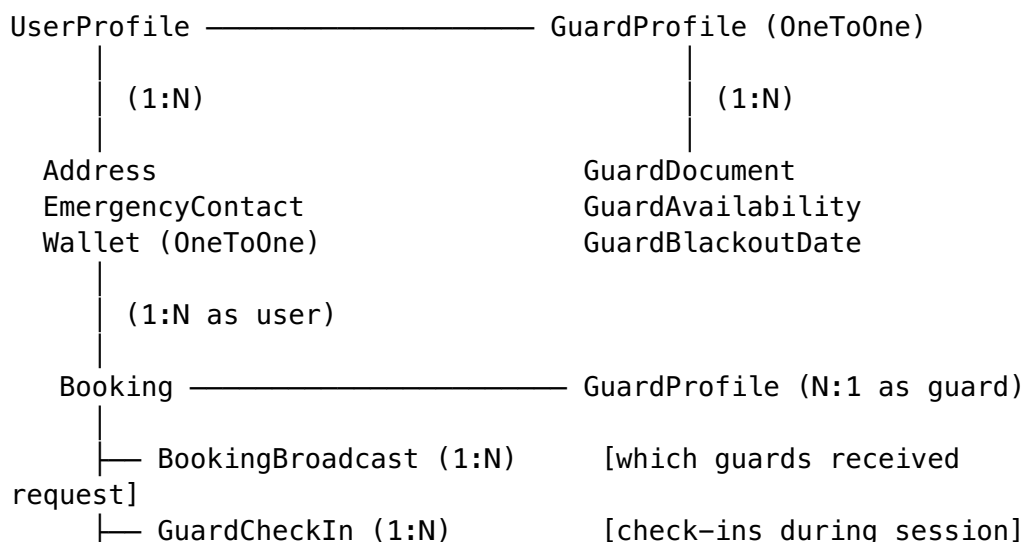
# Safety
sos_alerts = models.PositiveIntegerField(default=0)
incidents_filed = models.PositiveIntegerField(default=0)

created_at = models.DateTimeField(auto_now_add=True)
updated_at = models.DateTimeField(auto_now=True)

class Meta:
    db_table = 'analytics_daily_stats'
    ordering = ['-date']

```

11. Entity Relationship Overview



— LocationSnapshot (1:N)	[tracking data]
— Transaction (1:N)	[payments]
— SOSAlert (1:N)	[safety events]
— Incident (1:N)	[incident reports]
— Review (1:1)	[post-session review]

12. Database Indexes & Performance

```

-- PostGIS spatial index (auto-created by Django GIS)
-- Enables fast "guards within X km" queries
CREATE INDEX guards_profile_location_gist
ON guards_profile USING GIST (current_location);

-- Partial index: only active, online guards
-- (Most proximity queries only care about available guards)
CREATE INDEX guards_active_online
ON guards_profile (guard_type, average_rating DESC)
WHERE verification_status = 'ACTIVE' AND is_online = TRUE;

-- Booking queries by status (most common admin query)
CREATE INDEX bookings_active_sessions
ON bookings_booking (scheduled_start, user_id, guard_id)
WHERE status IN ('ACTIVE', 'EN_ROUTE', 'ARRIVED');

-- SOS alerts: only non-resolved (small, always fast)
CREATE INDEX sos_open_alerts
ON sos_alert (created_at DESC)
WHERE status != 'RESOLVED' AND status != 'FALSE_ALARM';

-- Location snapshots: per-booking time-range (session replay)
CREATE INDEX tracking_booking_time
ON tracking_location_snapshot (booking_id, timestamp DESC);

-- Wallet transactions for a user (paginated list)
CREATE INDEX payments_wallet_transactions
ON payments_transaction (wallet_id, created_at DESC);

-- Unread in-app notifications
CREATE INDEX notifications_unread
ON notifications_log (recipient_id, created_at DESC)
WHERE is_read = FALSE AND channel = 'IN_APP';

```

13. Migrations Strategy

Running Migrations

```
# Create new migration after model changes
python manage.py makemigrations <app_name>

# Apply all pending migrations
python manage.py migrate

# Show migration status
python manage.py showmigrations

# Squash migrations (for clean production history)
python manage.py squashmigrations <app_name> 0001 0010
```

Rules for Production Migrations

1. **Never delete a column directly** — first deploy code that stops using it, then remove in a later migration.
2. **Adding nullable columns** is safe and zero-downtime. Adding NOT NULL columns requires a default or a data migration first.
3. **Index creation** — use `db_index=True` or `Meta.indexes`. For large tables, use `CREATE INDEX CONCURRENTLY` in a RunSQL migration to avoid locking.
4. **RunSQL for PostGIS** — enable extensions in migrations:

```
# apps/users/migrations/0001_initial.py (partial)
from django.db import migrations

class Migration(migrations.Migration):
    operations = [
        migrations.RunSQL(
            sql='CREATE EXTENSION IF NOT EXISTS postgis;',
            reverse_sql='DROP EXTENSION IF EXISTS postgis;',
        ),
        # ... rest of operations
    ]
```

1. **Data migrations** for backfills:

```
# Example: backfill wallet for existing users
from django.db import migrations

def create_wallets(apps, schema_editor):
    UserProfile = apps.get_model('users', 'UserProfile')
    Wallet = apps.get_model('payments', 'Wallet')
    for user in UserProfile.objects.filter(role='USER'):
        Wallet.objects.get_or_create(user=user)

class Migration(migrations.Migration):
```

```
operations = [  
    migrations.RunPython(create_wallets,  
        migrations.RunPython.noop),  
]
```